

Dhirubhai Ambani University

(Formerly known as DA-IICT)

Topic: Error and Exceptions

Dr. Ankit Vijayvargiya

Assistant Professor

Room No. 4205, Faculty Block 4

Email: [ankit_Vijayvargiya\[at\]dau.ac.in](mailto:ankit_Vijayvargiya[at]dau.ac.in)

Errors and Exceptions

An error is a problem in the program that prevents it from running as expected.

These are two main types:

1. Syntax Errors (Compile-time error)

Example:

```
if X > 5
    print("x is greater than 5")
```

```
: if X > 5
    print('x is greater than 5')
```

```
Cell In[7], line 1
    if X > 5
        ^
SyntaxError: expected ':'
```

Fix Code

2. Logical Errors (Runtime error)

i) This occurs while the program is running.

ii) They are technically exceptions

Exception: An exception is an event, which occurs during the execution of a program, that disrupts the normal flow of program.

```
: a=5
  b=0
  res=a/b
```

```
-----
ZeroDivisionError                                Traceback (most recent call last)
Cell In[2], line 3
      1 a=5
      2 b=0
----> 3 res=a/b

ZeroDivisionError: division by zero
```

Whenever this type of runtime error occurs, Python create an exception object. If not handled properly, it prints a traceback for that error along with details about why it occurred.

Exception handling is the process of **detecting, catching, and managing errors (exceptions)** that occur during the execution of a program, so that the program doesn't crash unexpectedly.

Errors and Exceptions

Four keywords:

try

except

else

finally

try: In which those lines or codes are placed which might contain errors. So, we have to handle because our program will not close abruptly.

except: This is the block in which statements handle the exceptions are written.

else: The statements inside the else block are executed when there is no exception found in try block.

finally: The statement(s) inside finally block are executed irrespective of whether any exception is found in try block or not.

Errors and Exceptions

Exception Handling:

To handle exceptions and to call code when an exception occurs, we can use a **try/except** statement.

- The try block contains code that might throw an exception.
- If that exception occurs, the code in the try block stops being executed, and the code in the except block is executed.
- If no error occurs, the code in the except block doesn't execute.

```
a=float(input("Enter first number"))
b=float(input("Enter second number"))
c=a/b
print(c)
```

```
Enter first number 5
Enter second number 2
2.5
```

```
a=float(input("Enter first number"))
b=float(input("Enter second number"))
c=a/b
print(c)
```

```
Enter first number 5
Enter second number 0
```

```
-----
ZeroDivisionError                                Traceback
Cell In[2], line 3
      1 a=float(input("Enter first number"))
      2 b=float(input("Enter second number"))
----> 3 c=a/b
      4 print(c)
```

Errors and Exceptions

Exception Handling:

To handle exceptions and to call code when an exception occurs, we can use a **try/except** statement.

- The try block contains code that might throw an exception.
- If that exception occurs, the code in the try block stops being executed, and the code in the except block is executed.
- If no error occurs, the code in the except block doesn't execute.

```
c=0
a=float(input("Enter first number"))
b=float(input("Enter second number"))
try:
    c=a/b
except:
    print("Error")
print("Result=",c)
```

```
Enter first number 5
Enter second number 2
Result= 2.5
```

```
c=0
a=float(input("Enter first number"))
b=float(input("Enter second number"))
try:
    c=a/b
except:
    print("Error")
print("Result=",c)
```

```
Enter first number 5
Enter second number 0
Error
Result= 0
```

Errors and Exceptions

Exception Handling:

```
try:
    a=float(input("Enter first number"))
    b=float(input("Enter second number"))
    c=a/b
except ZeroDivisionError as f:
    print("Error:", f)
print("Result=",c)
```

```
Enter first number 5
Enter second number 0
Error: float division by zero
Result= 0
```

```
c=0
try:
    a=float(input("Enter first number"))
    b=float(input("Enter second number"))
    c=a/b
except ZeroDivisionError as f:
    print("Error:", f)
print("Result=",c)
```

```
Enter first number 5
Enter second number hi
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[29], line 4
      2 try:
      3     a=float(input("Enter first number"))
----> 4     b=float(input("Enter second number"))
      5     c=a/b
      6 except ZeroDivisionError as f:

ValueError: could not convert string to float: 'hi'
```

Errors and Exceptions

Exception Handling:

```
c=0
a=float(input("Enter first number"))
b=float(input("Enter second number"))
try:
    c=a/b
except Exception as f:
    print("Error:",f)
print("Result=",c)
```

```
Enter first number 5
Enter second number 0
Error: float division by zero
Result= 0
```

```
try:
    a=float(input("Enter first number"))
    b=float(input("Enter second number"))
    c=a/b
except Exception as f:
    print("Error:", f)
print("Result=",c)
```

```
Enter first number 5
Enter second number hi
Error: could not convert string to float: 'hi'
Result= 0
```

Catch any standard run-time error:

- ZeroDivisionError
- ValueError
- TypeError
-

Errors and Exceptions

Nested Try Block:

```
c=0
try:
    a=float(input("Enter first number"))
    b=float(input("Enter second number"))
    try:
        c=a/b
    except ZeroDivisionError as f:
        print("Error:",f)
except:
    print("Invalid Input")

print("Result=",c)
```

```
Enter first number 5
Enter second number 0
Error: float division by zero
Result= 0
```

```
c=0
try:
    a=float(input("Enter first number"))
    b=float(input("Enter second number"))
    try:
        c=a/b
    except ZeroDivisionError as f:
        print("Error:",f)
except:
    print("Invalid Input")

print("Result=",c)
```

```
Enter first number 5
Enter second number Hi
Invalid Input
Result= 0
```

Errors and Exceptions

A try statement can have multiple different blocks to handle different exceptions.

```
c=0

try:
    a=float(input("Enter first number"))
    b=float(input("Enter second number"))
    c=a/b
except ZeroDivisionError as e:
    print("Error=", e)
except ValueError as e:
    print("Error:", e)

print("Result=",c)
```

```
Enter first number 5
Enter second number 0
Error= float division by zero
Result= 0
```

```
: c=0

try:
    a=float(input("Enter first number"))
    b=float(input("Enter second number"))
    c=a/b
except ZeroDivisionError as e:
    print("Error=", e)
except ValueError as e:
    print("Error:", e)

print("Result=",c)
```

```
Enter first number 5
Enter second number hi
Error: could not convert string to float: 'hi'
Result= 0
```

Errors and Exceptions

Multiple exceptions can also be put into a single except block using parentheses, so the except block handles all of them.

```
c=0

try:
    a=float(input("Enter first number"))
    b=float(input("Enter second number"))
    c=a/b
except (ZeroDivisionError, ValueError) as e:
    print("Error=", e)

print("Result=",c)
```

```
Enter first number 5
Enter second number 0
Error= float division by zero
Result= 0
```

```
c=0

try:
    a=float(input("Enter first number"))
    b=float(input("Enter second number"))
    c=a/b
except (ZeroDivisionError, ValueError) as e:
    print("Error=", e)

print("Result=",c)
```

```
Enter first number 5
Enter second number hi
Error= could not convert string to float: 'hi'
Result= 0
```

Errors and Exceptions

Raising Exception: raise is used to manually create (throw) an exception.

```
c=0
try:
    a=float(input("Enter first number"))
    b=float(input("Enter second number"))
    if b==0:
        raise ZeroDivisionError("Number can not be zero")
    c=a/b
except ZeroDivisionError as e:
    print("Error=", e)

print("Result=",c)
```

```
Enter first number 5
Enter second number 0
Error= Number can not be zero
Result= 0
```

```
c=0
try:
    a=float(input("Enter first number"))
    b=float(input("Enter second number"))
    if b==0:
        raise ZeroDivisionError("Number can not be zero")
    c=a/b
except ZeroDivisionError as e:
    print("Error=", e)

print("Result=",c)
```

```
Enter first number 5
Enter second number hi
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[40], line 4
      2 try:
      3     a=float(input("Enter first number"))
----> 4     b=float(input("Enter second number"))
      5     if b==0:
      6         raise ZeroDivisionError("Number can not be zero")

ValueError: could not convert string to float: 'hi'
```

Errors and Exceptions

Raising Exception: raise is used to manually create (throw) an exception.

```
: c=0
try:
    a=float(input("Enter first number"))
    b=float(input("Enter second number"))
    if b==0:
        raise ZeroDivisionError("Number can not be zero")
    c=a/b
except ZeroDivisionError as e:
    print("Error=", e)

print("Result=",c)
```

Enter first number 5
Enter second number 0
Error= Number can not be zero
Result= 0

```
c=0
try:
    a=float(input("Enter first number"))
    b=float(input("Enter second number"))
    if b==0:
        raise Exception("Number can not be zero")
    c=a/b
except Exception as e:
    print("Error=", e)

print("Result=",c)
```

Enter first number 5
Enter second number 0
Error= Number can not be zero
Result= 0

Errors and Exceptions

```
c=0
try:
    a=float(input("Enter first number"))
    b=float(input("Enter second number"))
    if b==0:
        raise Ankit("Number can not be zero")
    c=a/b
except Ankit as e:
    print("Error=", e)

print("Result=",c)
```

Enter first number 5
Enter second number 0

```
-----
NameError                                Traceback (most recent call last)
Cell In[42], line 6
      5 if b==0:
----> 6     raise Ankit("Number can not be zero")
      7 c=a/b
```

NameError: name 'Ankit' is not defined

During handling of the above exception, another exception occurred:

```
NameError                                Traceback (most recent call last)
Cell In[42], line 8
      6     raise Ankit("Number can not be zero")
      7     c=a/b
----> 8 except Ankit as e:
      9     print("Error=", e)
     11 print("Result=",c)
```

NameError: name 'Ankit' is not defined

User-defined Exception:

```
: c=0
class Ankit(Exception):
    pass
try:
    a=float(input("Enter first number"))
    b=float(input("Enter second number"))
    if b==0:
        raise Ankit("Number can not be zero")
    c=a/b
except Ankit as e:
    print("Error=", e)

print("Result=",c)
```

Enter first number 5
Enter second number 0
Error= Number can not be zero
Result= 0

Errors and Exceptions

User-defined Exception:

```
class Ankit(Exception):
    def __init__(self):
        print("Number can not be zero")

c=0
try:
    a=float(input("Enter first number: "))
    b=float(input("Enter second number: "))
    if b==0:
        raise Ankit()
    c=a/b
except Ankit:
    pass
print("Result =", c)
```

```
Enter first number: 5
Enter second number: 0
Number can not be zero
Result = 0
```

```
class Ankit(Exception):
    def __init__(self, value):
        self.value = value

c=0
try:
    a=float(input("Enter first number: "))
    b=float(input("Enter second number: "))
    if b==0:
        raise Ankit("Number can not be zero")
    c=a/b
except Ankit as e:
    print("Error =", e.value)

print("Result =", c)
```

```
Enter first number: 5
Enter second number: 0
Error = Number can not be zero
Result = 0
```

Errors and Exceptions

else:

Else runs only when no exception occurs in the try.

```
: c=0
try:
    a=float(input("Enter first number"))
    b=float(input("Enter second number"))
    c=a/b
except:
    print("Number can not be zero")
else:
    print("Result=",c)
```

```
Enter first number 5
Enter second number 0
Number can not be zero
```

```
c=0
try:
    a=float(input("Enter first number"))
    b=float(input("Enter second number"))
    c=a/b
except:
    print("Number can not be zero")
else:
    print("Result=",c)
```

```
Enter first number 5
Enter second number 2
Result= 2.5
```

Errors and Exceptions

finally:

It always runs whether an error occurs, no error, return used, or an exception raised.

```
try:
    a = int(input("Enter number: "))
    b = int(input("Enter number: "))
    c = a / b

except Exception as e:
    print("Error:", e)

else:
    print("Result =", c)

finally:
    print("Program ended")
```

```
Enter number: 5
Enter number: 0
Error: division by zero
Program ended
```

```
try:
    a = int(input("Enter number: "))
    b = int(input("Enter number: "))
    c = a / b

except Exception as e:
    print("Error:", e)

else:
    print("Result =", c)

finally:
    print("Program ended")
```

```
Enter number: 5
Enter number: 2
Result = 2.5
Program ended
```

Errors and Exceptions

Four keywords:

try

except

else

finally

try: In which those lines or codes are placed that might contain errors. So, we have to handle it because our program will not close abruptly.

Except: This is the block in which statements that handle the exceptions are written.

else: The statements inside the else block are executed when there is no exception found in the try block.

Finally: The statement(s) inside the finally block are executed irrespective of whether any exception is found in the try block or not.

```
try:
    # risky code
except:
    # runs if error
else:
    # runs if no error
finally:
    # ALWAYS runs
```

Errors and Exceptions

Some Imp points:

- A single try statements can have multiple except statements. This is useful when the try block contains statements that throw different types of exceptions.

```
try:  
    "body of try"  
except some_exception:  
    "body of except"  
except some_another_exception:  
    "body of except"
```

- You can provide a generic except clause which handles any exceptions.

```
try:  
    a=10/0  
except:  
    print("An error occurred")
```

- You can have only at most one else block and at most one finally block

```
try:  
    "body of try"  
except some_exception:  
    "body of except"  
except some_another_exception:  
    "body of except"  
else:  
    "body of else"  
finally:  
    "body of finally"
```

Exercise

Write a Python program that accepts marks for n subjects and calculates the total and percentage.

Requirements:

- Ask the user for the number of subjects.
- Accept marks for each subject.
- Handle ValueError when the user enters non-numeric data.
- Handle ZeroDivisionError when the number of subjects is 0.
- Handle IndexError if you store marks in a list and attempt to access an invalid index.
- Use an exception object with as e to display the error message.
- Print the total, percentage, and grade when the input is valid.

```
Enter student name: Ram
Enter number of subjects: 5
Enter marks for subject 1: 80
Enter marks for subject 2: 90
Enter marks for subject 3: 72
Enter marks for subject 4: 78
Enter marks for subject 5: 88

--- Student Result ---
Name: Ram
Total: 408.0
Percentage: 81.6
Grade: A
```

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
    def calculate_result(self):
        total = sum(self.marks)
        percentage = total / len(self.marks)
        if percentage >= 90:
            grade = "A+"
        elif percentage >= 80:
            grade = "A"
        elif percentage >= 70:
            grade = "B"
        elif percentage >= 60:
            grade = "C"
        elif percentage >= 50:
            grade = "D"
        else:
            grade = "F"

        print("\n--- Student Result ---")
        print("Name:", self.name)
        print("Total:", total)
        print("Percentage:", percentage)
        print("Grade:", grade)

# Creating object
name = input("Enter student name: ")
n = int(input("Enter number of subjects: "))
marks = []
for i in range(n):
    mark = float(input(f"Enter marks for subject {i + 1}: "))
    marks.append(mark)
student1 = Student(name, marks)
student1.calculate_result()
```

Exercise

Write a Python program that accepts marks for n subjects and calculates the total and percentage.

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
    def calculate_result(self):
        total = sum(self.marks)
        percentage = total / len(self.marks)

        if percentage >= 90:
            grade = "A+"
        elif percentage >= 80:
            grade = "A"
        elif percentage >= 70:
            grade = "B"
        elif percentage >= 60:
            grade = "C"
        elif percentage >= 50:
            grade = "D"
        else:
            grade = "F"
        print("\n--- Student Result ---")
        print("Name:", self.name)
        print("Total:", total)
        print("Percentage:", percentage)
        print("Grade:", grade)
```

```
try:
    name = input("Enter student name: ")
    n = int(input("Enter number of subjects: "))
    if n <= 0:
        raise ValueError("Number of subjects must be greater than 0")
    marks = []
    for i in range(n):
        mark = float(input(f"Enter marks for subject {i + 1}: "))
        if mark < 0 or mark > 100:
            raise ValueError("Marks must be between 0 and 100")
        marks.append(mark)
    # Creating object
    student1 = Student(name, marks)
    # Calling method
    student1.calculate_result()
except ValueError as e:
    print("Value Error:", e)
except ZeroDivisionError as e:
    print("Zero Division Error:", e)
except Exception as e:
    print("Unexpected Error:", e)
```